



AULA 04

COMPLEXIDADE DE ALGORITMOS

Estruturas Lineares — Parte 4

Contagem de operações e as notações O , Ω e Θ

Estruturas de Dados — UniLasalle EAD

Grau 1 · Aula 4 de 4 · 2026



DESEMPENHO DE ALGORITMOS

OBJETIVOS DE APRENDIZAGEM

1

Contar as operações de um algoritmo e expressar o custo como uma função $T(n)$.

2

Interpretar funções, crescimento, logaritmo, exponencial, fatorial e somatórios.

3

Definir e aplicar as notações O , Ω e Θ .

4

Analisar **melhor, pior e caso médio** — com o Insertion Sort como estudo de caso.

5

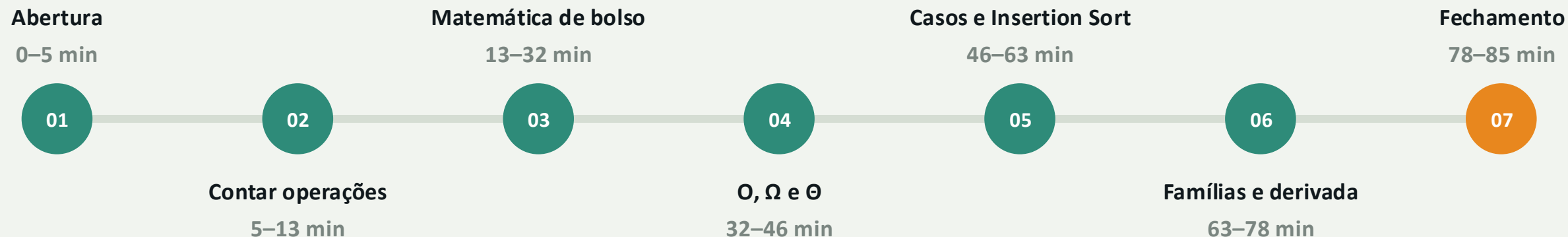
Comparar famílias de funções e usar a **derivada** para entender a taxa de crescimento.

$T(n)$

$O \cdot \Omega \cdot \Theta$

contar, simplificar
e comparar
custos

AGENDA DA AULA



IDE ONLINE

onlineide.pro/playground/python
onlineide.pro/playground/c

EXERCÍCIOS

pause o vídeo e tente
responda no chat

APOSTILA

o selo no canto de cada slide
indica a página que aprofunda

REPOSITÓRIO

[github.com/ProfessorFilipo/
EstruturasDados](https://github.com/ProfessorFilipo/EstruturasDados) → aula04-
complexidade

AFINAL, O QUE É $O(1)$?

Operação (tabela da Aula 1)	Contígua (array)	Encadeada (lista)
Acesso por índice	$O(1)$	$O(n)$
Inserir/remover no início	$O(n)$	$O(1)$
Inserir/remover no fim	$O(1)^*$	$O(1)^{**}$

* com capacidade suficiente ** com ponteiro para o fim

Pergunta da Aula 2: por que manter um ponteiro para o fim permite enfileirar em $O(1)$? Hoje ela ganha uma resposta formal.

Usamos esses símbolos desde a Aula 1.

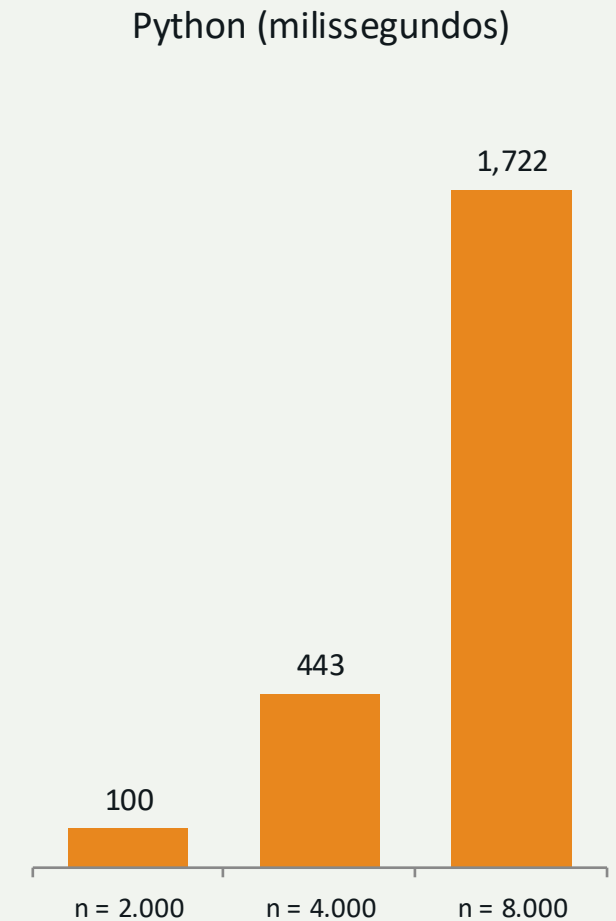
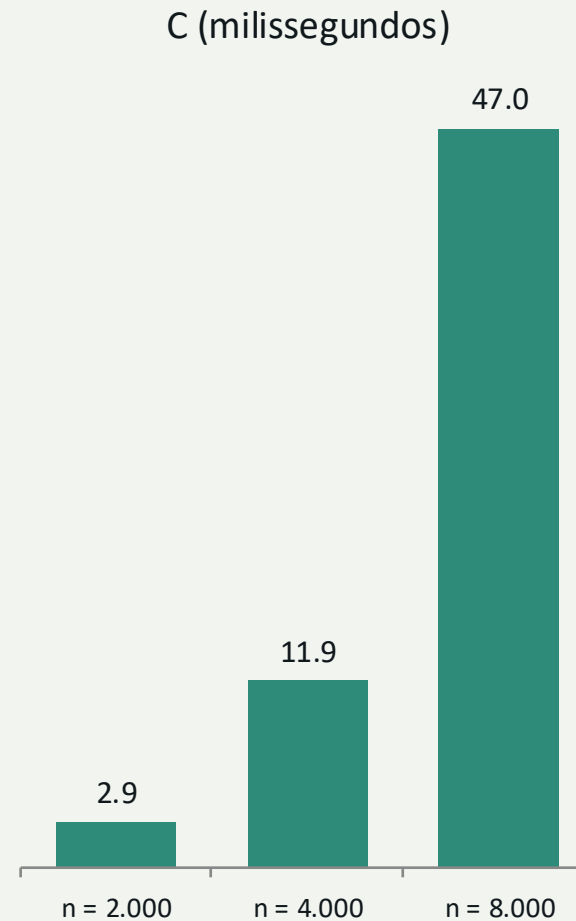
Hoje vamos entender:

- **o que** eles significam
- **como** calculá-los
- **por que** eles importam mais que o computador

CRONOMETRAR ENGANA

- O tempo depende de muita coisa além do algoritmo: processador, linguagem, outros programas abertos.
- Medimos o mesmo Insertion Sort (vetor em ordem inversa) em **C** e em **Python**.
- Python $\approx$ 35 vezes mais lento — mas, nas duas linguagens, **dobrar n multiplica o tempo por ≈ 4** .

O número de segundos muda de máquina para máquina. **A forma do crescimento, não.** Por isso vamos contar passos.



CONTANDO OPERAÇÕES

```
int soma(const int v[], int n) {  
    int total = 0;           /* 1 vez */  
    for (int i = 0; i < n; i++) { /* 1 | n + 1 | n */  
        total = total + v[i]; /* n vezes */  
    }  
    return total;           /* 1 vez */  
}
```

c

Modelo de custo: cada instrução simples executada custa **1 passo** — uma atribuição, um teste do laço, um incremento, um comando do corpo ou um return.

Por que $n + 1$ testes? O último teste dá falso e encerra o laço.

$$T(n) = 1 + 1 + (n + 1) + n + n + 1$$

$$T(n) = 3n + 4$$

n	$T(n) = 3n + 4$
10	34
100	304
1.000	3.004

Confira ao vivo: [python/contagem_operacoes.py](#)

EXERCÍCIO 1

EXERCÍCIO 1

PAUSE E TENTE · RESPONDA NO CHAT · CERCA DE 2 MIN

```
int buscar(const int dados[], int tamanho, int valor) {  
    for (int i = 0; i < tamanho; i++) {  
        if (dados[i] == valor) {  
            return i;  
        }  
    }  
    return -1;  
}
```

c

É o `lista_buscar()` da Aula 1, com um vetor comum.

Use o mesmo modelo de custo do slide anterior.

1. Quantos passos a função executa no **pior caso** (o valor não está no vetor)? Escreva **$T(n)$** .

2. Quanto vale **$T(100)$** ?

RESPOSTA 1

RESPOSTA 1

```
int buscar(const int dados[], int tamanho, int valor) {  
    for (int i = 0; i < tamanho; i++) { /* 1 | n + 1 | n */  
        if (dados[i] == valor) { /* n vezes */  
            return i;  
        }  
    }  
    return -1; /* 1 vez */  
}
```

c

$$1 + (n + 1) + n + n + 1$$

$$T(n) = 3n + 3$$

$$T(100) = 303 \text{ passos}$$

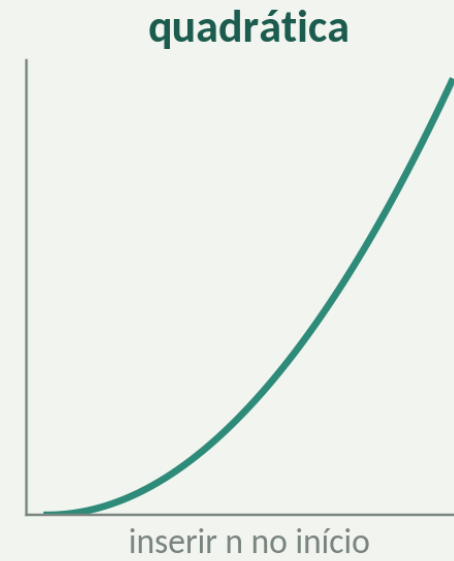
E se o valor estiver na posição 0? $i = 0$, um teste, uma comparação e o return: **4 passos**, qualquer que seja n . O mesmo algoritmo, com o mesmo n , pode custar 4 ou $3n + 3$ — é o **melhor caso** (slide 28).

FUNÇÕES E CRESCIMENTO

$n = 5$ → $T(n) = 3n + 4$ → $T(5) = 19$

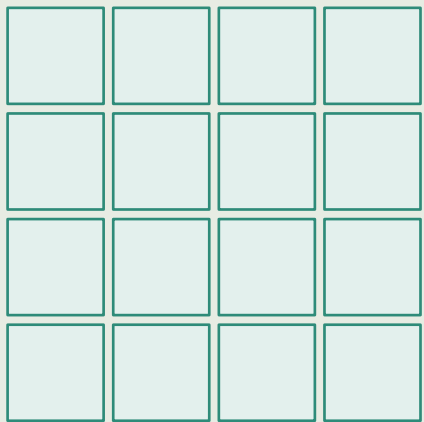
entrada: o tamanho (o tamanho() da Lista)

saída: o custo — passos, comparações...



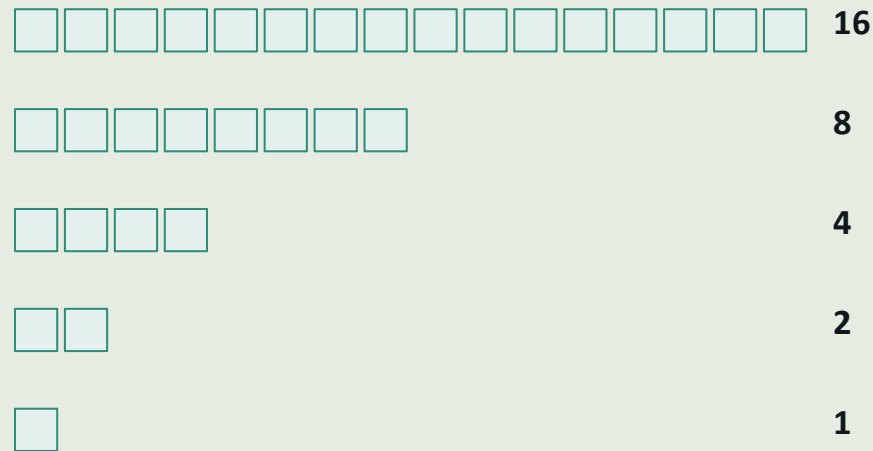
Crescente: aumentou n , o custo nunca diminui. O que muda é a **velocidade**: dobrou n → custo igual, dobrado ou quadruplicado.

POTÊNCIAS, RAÍZES E LOG



$n^2 = 4 \times 4 = 16$ casas
 $\sqrt{16} = 4$ (o lado)

Um tabuleiro de Campo Minado, uma matriz numa lista contígua...



$16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$: 4 divisões
 $\log_2 16 = 4$, porque $2^4 = 16$

n	$\log_2 n$
8	3
1.024	10
1.000.000	≈ 20

Adivinhe de 1 a 100 dividindo ao meio: no máximo **7 palpites** ($2^7 = 128$).

O logaritmo cresce **muito devagar**: de mil para um milhão, só vai de 10 para 20. E a base não importa em O.

EXPONENCIAL E FATORIAL

2^n : dobra a cada +1 — um Conjunto com n elementos tem 2^n subconjuntos (Aula 3)

$\emptyset$

$\{A\}$

$\{B\}$

$\{C\}$

$\{A, B\}$

$\{A, C\}$

$\{B, C\}$

$\{A, B, C\}$

$\{A, B, C\}$: $2^3 = 8$ subconjuntos

$n!$: todas as ordens — de quantas formas 3 trabalhos chegam à fila de impressão (Aula 2)?

101 102 103

101 103 102

102 101 103

102 103 101

103 101 102

103 102 101

$3! = 3 \times 2 \times 1 = 6$ ordens

n	5	10	20
2^n	32	1.024	≈ 1 milhão
$n!$	120	3.628.800	$\approx 2,4 \times 10^{18}$

EXERCÍCIO 2

EXERCÍCIO 2

PAUSE E TENTE • RESPONDA NO CHAT • CERCA DE 2 MIN

a Quanto vale $\log_2 64$? (quantas vezes dá para dividir 64 ao meio até chegar a 1?)

b Quantos e quais são os subconjuntos de $\{A, B, C\}$?

c De quantas formas **4 trabalhos** podem chegar à fila de impressão?

RESPOSTA 2

RESPOSTA 2

6

$64 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$: seis divisões, porque $2^6 = 64$.

8

$\emptyset, \{A\}, \{B\}, \{C\}, \{A, B\}, \{A, C\}, \{B, C\}, \{A, B, C\}$ — cada elemento entra ou não: $2^3 = 8$.

24

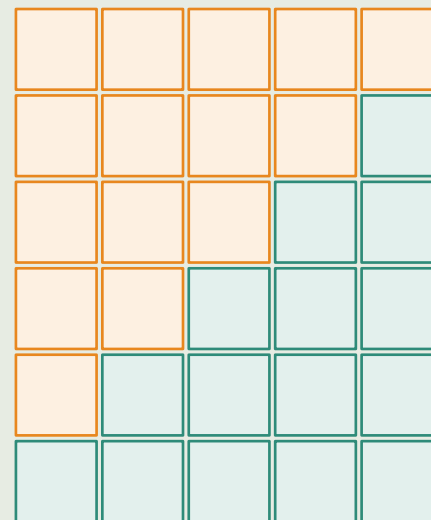
$4! = 4 \times 3 \times 2 \times 1 = 24$ ordens. Com 20 trabalhos, seriam $\approx 2,4 \times 10^{18}$.

SOMATÓRIOS E GAUSS

$$1 + 2 + 3 + \dots + n = n(n + 1) / 2$$

Σi , com i indo de 1 até n — "a soma de Gauss"

Gauss, criança, somou 1 a 100 em segundos: $100 \times 101 / 2 = 5.050$.



escada verde:

$1 + 2 + 3 + 4 + 5$

escada laranja:

a mesma, virada

juntas: $5 \times 6 = 30$

uma só: **15**

Aula 1 — contando deslocamentos: inserir n elementos, sempre no início da lista contígua, desloca $0 + 1 + 2 + \dots + (n - 1) = n(n - 1)/2$ elementos. Para $n = 100$: **4.950**.

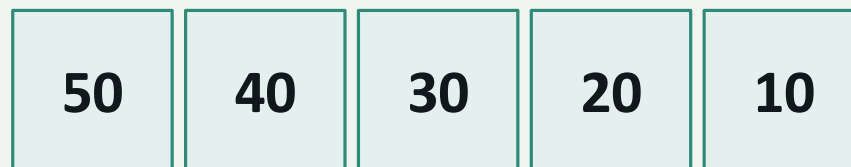
EXERCÍCIO 3

EXERCÍCIO 3

PAUSE E TENTE · RESPONDA NO CHAT · CERCA DE 2 MIN

Inserimos **10, 20, 30, 40 e 50**, um de cada vez, **sempre no início** de uma lista contígua vazia.

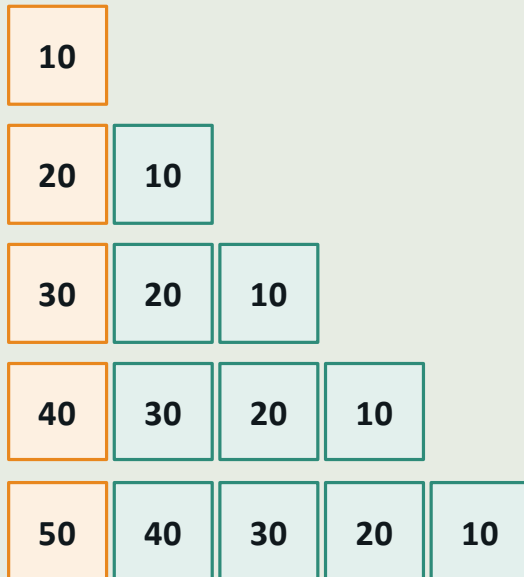
- (a) Quantos deslocamentos acontecem ao todo?
- (b) E se fossem **n = 100** inserções?



estado final: cada novo elemento empurra todos os anteriores

RESPOSTA 3

RESPOSTA 3



insere 10: **0 deslocamentos**

insere 20: **1 deslocamento**

insere 30: **2 deslocamentos**

insere 40: **3 deslocamentos**

insere 50: **4 deslocamentos**

$$(a) 0 + 1 + 2 + 3 + 4 = \mathbf{10}$$
$$= 5 \times 4 / 2$$

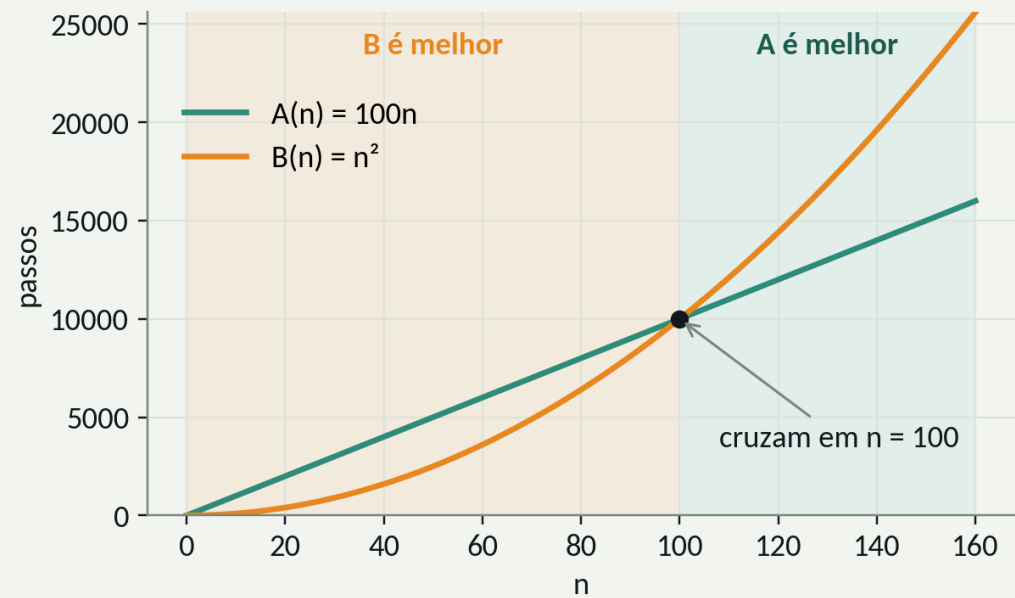
$$(b) 0 + 1 + \dots + 99$$
$$= 100 \times 99 / 2 = \mathbf{4.950}$$

TERMO DOMINANTE

n	$3n^2$	$5n$	2	parte do $3n^2$
10	300	50	2	85,2%
100	30.000	500	2	98,4%
1.000	3.000.000	5.000	2	99,8%

$T(n) = 3n^2 + 5n + 2$

- **Termo dominante:** o que cresce mais rápido acaba respondendo por quase todo o custo (carro + cafezinho).
- **Regras:** fique só com ele e despreze a constante: $3n^2 + 5n + 2 \rightarrow n^2$.
- Constantes mudam de máquina para máquina; a forma do crescimento, não.



Análise assintótica: para n pequeno, tanto faz; para n grande, quem cresce mais devagar sempre vence.

EXERCÍCIO 4

EXERCÍCIO 4

PAUSE E TENTE · RESPONDA NO CHAT · CERCA DE 2 MIN

Como cresce cada função?

- (a) $4n + 10$
- (b) $n^2/2 + 3n$
- (c) $5 + 2 \cdot \log_2 n$
- (d) $2^n + n^3$
- (e) 1.000

Dois algoritmos:

$$A(n) = 50n$$

$$B(n) = n^2$$

A partir de qual n o algoritmo **A** passa a ser melhor?

RESPOSTA 4

RESPOSTA 4

Função	Cresce como
$4n + 10$	n
$n^2/2 + 3n$	n^2
$5 + 2 \cdot \log_2 n$	$\log n$
$2^n + n^3$	2^n
1.000	constante

$$50n = n^2$$
$$\rightarrow 50 = n$$

A é melhor para $n > 50$.

A constante 50 assusta no início, mas perde a importância.

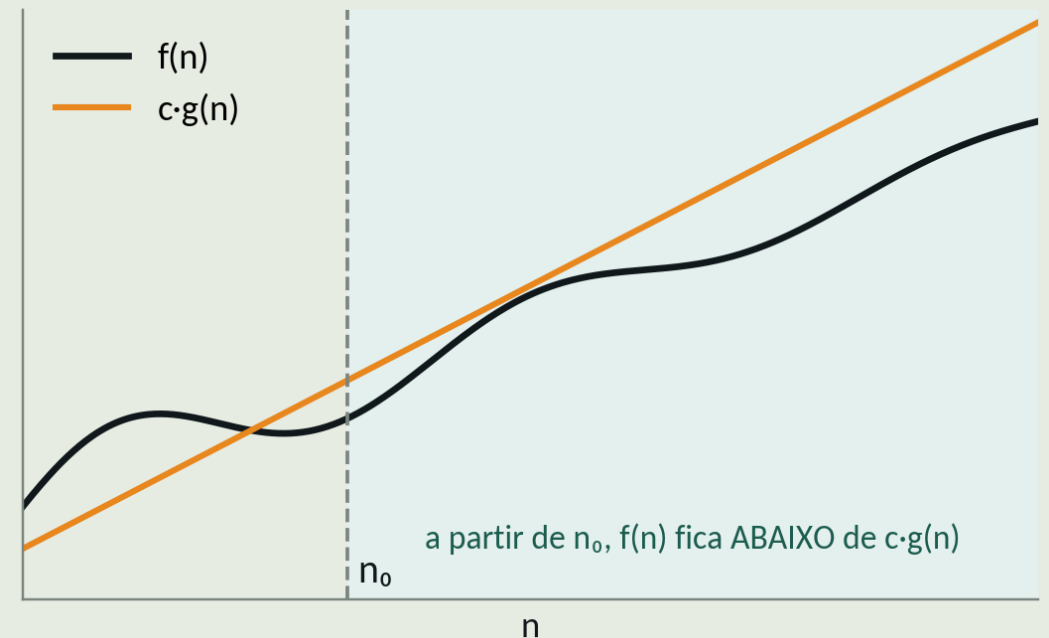
NOTAÇÃO O: O TETO

$f(n) = O(g(n))$ se existem constantes $c > 0$ e n_0 tais que

$f(n) \leq c \cdot g(n)$ para todo $n \geq n_0$

- Lê-se "**ó grande**": f não cresce mais rápido que g .
- c estica o teto (é o que permite desprezar constantes).
- n_0 : a partir de onde a regra vale; antes, pode falhar.

Como a capacidade da **Pilha contígua** (Aula 2): o topo nunca passa do teto.



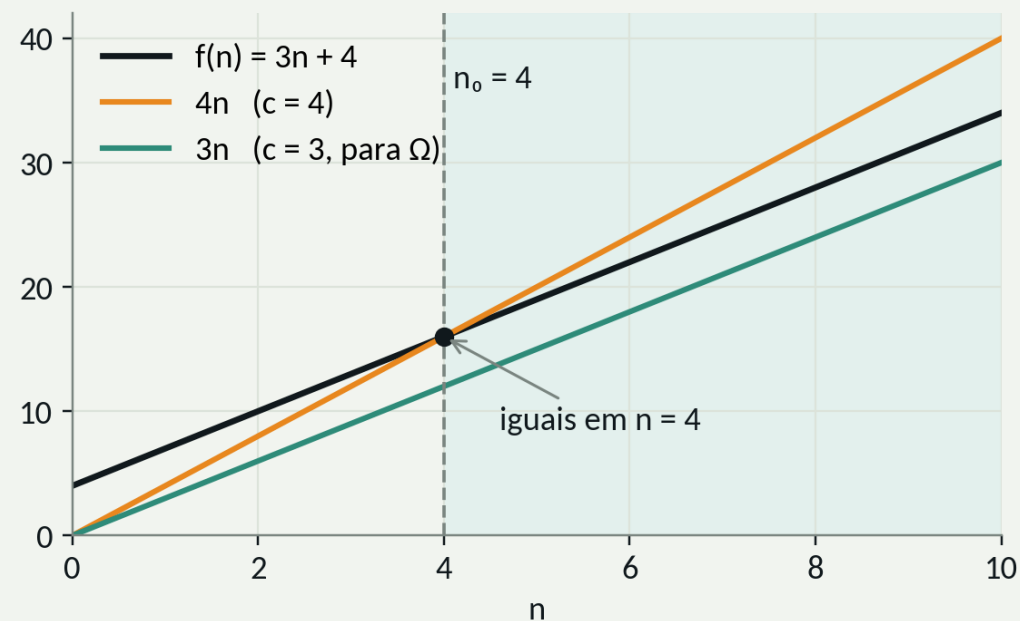
EXEMPLO GUIADO

Mostrar que $3n + 4 = O(n)$

- Escolha um c pouco maior que 3: $c = 4$.
- Resolva: $3n + 4 \leq 4n \rightarrow 4 \leq n$.
- Então vale para todo $n \geq 4$: $n_0 = 4$.

n	1	2	3	4	5	6
$3n + 4$	7	10	13	16	19	22
$4n$	4	8	12	16	20	24
vale?	não	não	não	sim	sim	sim

$3n + 4$ também é $O(n^2)$ — verdade, mas é um teto **folgado**. Buscamos sempre o mais justo.



EXERCÍCIO 5

EXERCÍCIO 5

PAUSE E TENTE · RESPONDA NO CHAT · CERCA DE 2 MIN

a

Mostre que $2n + 10 = O(n)$: encontre um par de constantes c e n_0 .

b

n^2 é $O(n)$? Justifique.

RESPOSTA 5

RESPOSTA 5

$$c = 3$$
$$n_0 = 10$$

$2n + 10 \leq 3n \rightarrow 10 \leq n$. Outro par válido: $c = 12, n_0 = 1$ (some os coeficientes: $2n + 10 \leq 2n + 10n$).

NÃO

Se fosse, $n^2 \leq c \cdot n$ para todo $n \geq n_0$, ou seja, $n \leq c$. Falha para qualquer $n > c$: nenhuma constante segura n^2 .

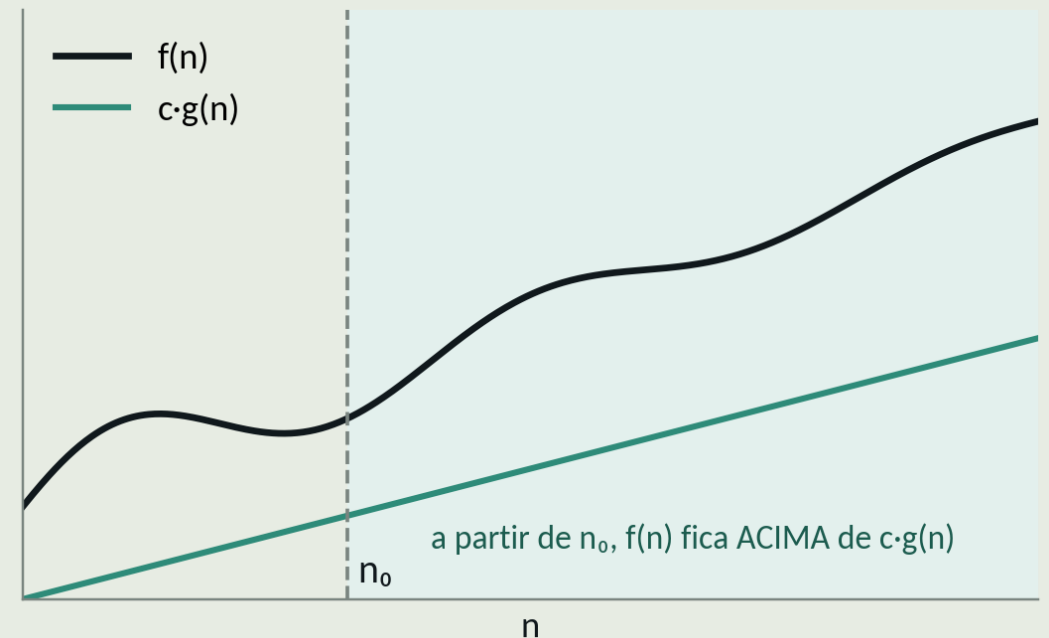
NOTAÇÃO Ω : O PISO

$f(n) = \Omega(g(n))$ se existem $c > 0$ e n_0 tais que

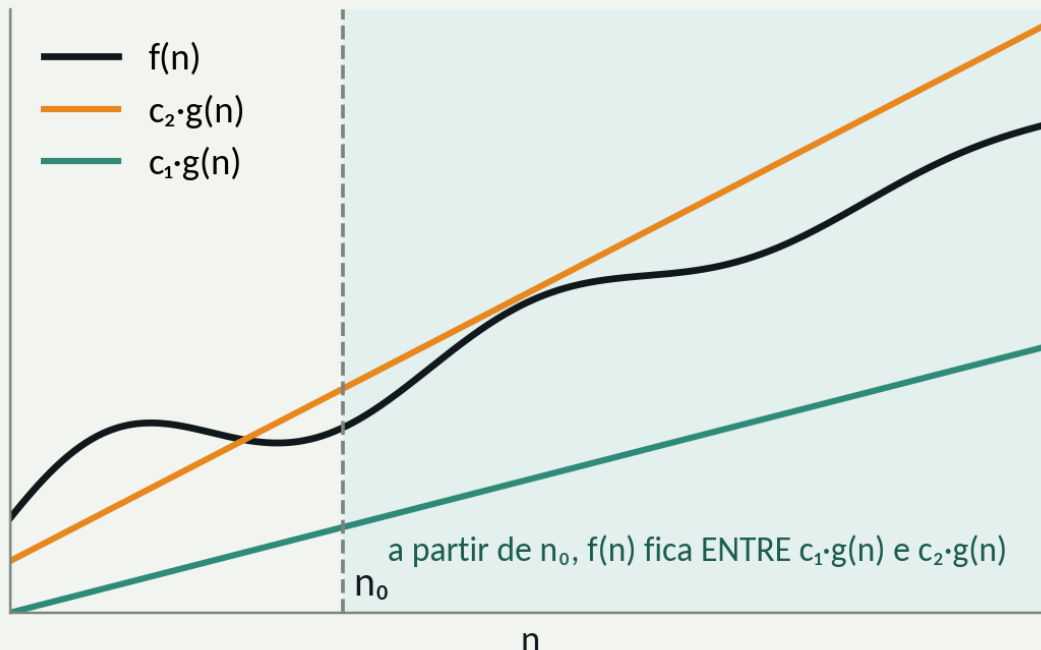
$f(n) \geq c \cdot g(n)$ para todo $n \geq n_0$

- Lê-se "**ômega grande**": um **piso** — f cresce pelo menos como g .
- Exemplo: $3n + 4 \geq 3n$ sempre $\rightarrow 3n + 4 = \Omega(n)$ ($c = 3$, $n_0 = 1$).

Ω também descreve problemas: achar o maior valor numa lista não ordenada exige olhar os n elementos. Todo algoritmo para isso é $\Omega(n)$.



NOTAÇÃO Θ : O EXATO



$f = \Theta(g)$ quando $f = O(g)$ e $f = \Omega(g)$:
 $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ para $n \geq n_0$

Notação	Lê-se	Ideia	Como comparar
O	ó grande	teto	$\leq$
Ω	ômega grande	piso	$\geq$
Θ	teta grande	exato	$=$

- Pilha: empilhar e desempilhar são $\Theta(1)$.
- percorrer() uma Lista é $\Theta(n)$.
- Inserir n no início da contígua: $\Theta(n^2)$.

EXERCÍCIO 6

EXERCÍCIO 6

PAUSE E TENTE · RESPONDA NO CHAT · CERCA DE 2 MIN

a

Mostre que $3n + 4 = \Theta(n)$: dê as constantes c_1 , c_2 e n_0 . (Use os slides 21 e 24.)

b

$n^2 + n$ é Θ de quê? Dê c_1 , c_2 e n_0 .

RESPOSTA 6

RESPOSTA 6

$$c_1 = 3$$

$$c_2 = 4$$

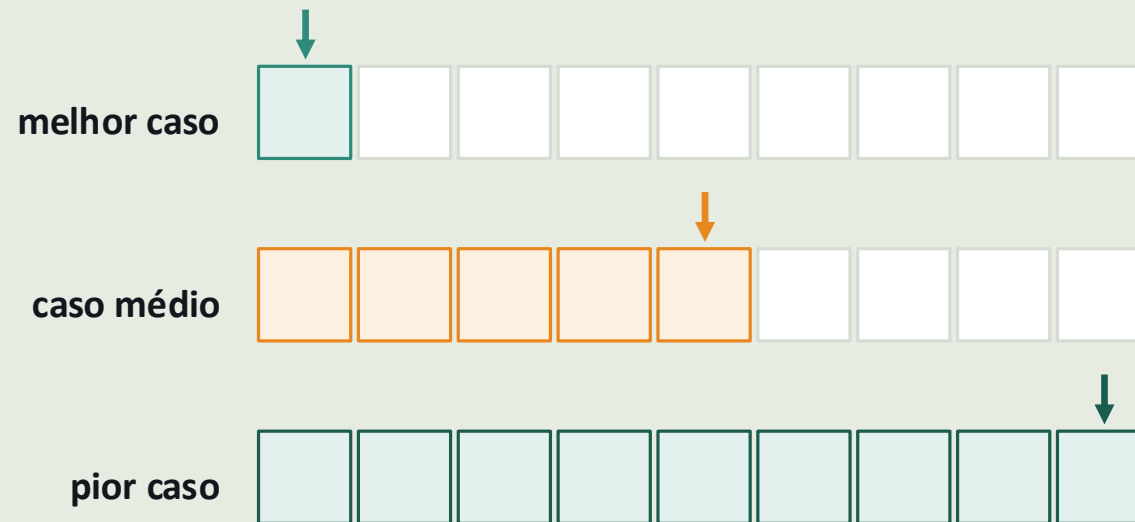
$$n_0 = 4$$

Piso: $3n + 4 \geq 3n$ (sempre). Teto: $3n + 4 \leq 4n$ (para $n \geq 4$). Juntando: **$3n \leq 3n + 4 \leq 4n$** para todo $n \geq 4$.

$$\Theta(n^2)$$

Para $n \geq 1$: **$n^2 \leq n^2 + n \leq 2n^2$** , com $c_1 = 1$, $c_2 = 2$ e $n_0 = 1$. Regra prática: um polinômio é Θ do seu maior expoente.

MELHOR, PIOR E MÉDIO



Caso	Comparações	Crescimento
Melhor	1	$\Theta(1)$
Médio	$(n + 1)/2$	$\Theta(n)$
Pior	n	$\Theta(n)$

Busca linear num vetor de 9 posições: as células coloridas são comparadas até achar o valor (seta).

Caso médio: $(1 + 2 + \dots + n) / n = (n + 1)/2$ — a soma de Gauss de novo. **Pior caso** é a garantia: o algoritmo nunca será mais lento que isso.

INSERTION SORT

Como **arrumar cartas na mão**: pegue a próxima (a **chave**), desloque para a direita as maiores e encaixe-a. É o **inserir(pos, valor)** da lista contígua da Aula 1.

```
def insertion_sort(v):  
    for i in range(1, len(v)):  
        chave = v[i]  
        j = i - 1  
        while j >= 0 and v[j] > chave:  
            v[j + 1] = v[j]    # desloca  
            j -= 1  
        v[j + 1] = chave      # encaixa
```

PYTHON

```
void insertion_sort(int v[], int n) {  
    for (int i = 1; i < n; i++) {  
        int chave = v[i];  
        int j = i - 1;  
        while (j >= 0 && v[j] > chave) {  
            v[j + 1] = v[j];    /* desloca */  
            j--;  
        }  
        v[j + 1] = chave;      /* encaixa */  
    }  
}
```

C

PASSO A PASSO

5	2	4	6	1	3
---	---	---	---	---	---

início: a parte ordenada é só o 5

2	5	4	6	1	3
---	---	---	---	---	---

passada 1, chave 2: **1** comparação, **1** deslocamento

2	4	5	6	1	3
---	---	---	---	---	---

passada 2, chave 4: **2** comparações, **1** deslocamento

2	4	5	6	1	3
---	---	---	---	---	---

passada 3, chave 6: **1** comparação, **0** deslocamentos

1	2	4	5	6	3
---	---	---	---	---	---

passada 4, chave 1: **4** comparações, **4** deslocamentos

1	2	3	4	5	6
---	---	---	---	---	---

passada 5, chave 3: **4** comparações, **3** deslocamentos

verde: parte ordenada **laranja:** onde a chave foi encaixada

total

12
comparações

9
deslocamentos

MELHOR x PIOR CASO

MELHOR CASO: VETOR ORDENADO

1	2	3	4	5	
1	2	3	4	5	1 comp., 0 desl.
1	2	3	4	5	1 comp., 0 desl.
1	2	3	4	5	1 comp., 0 desl.
1	2	3	4	5	1 comp., 0 desl.

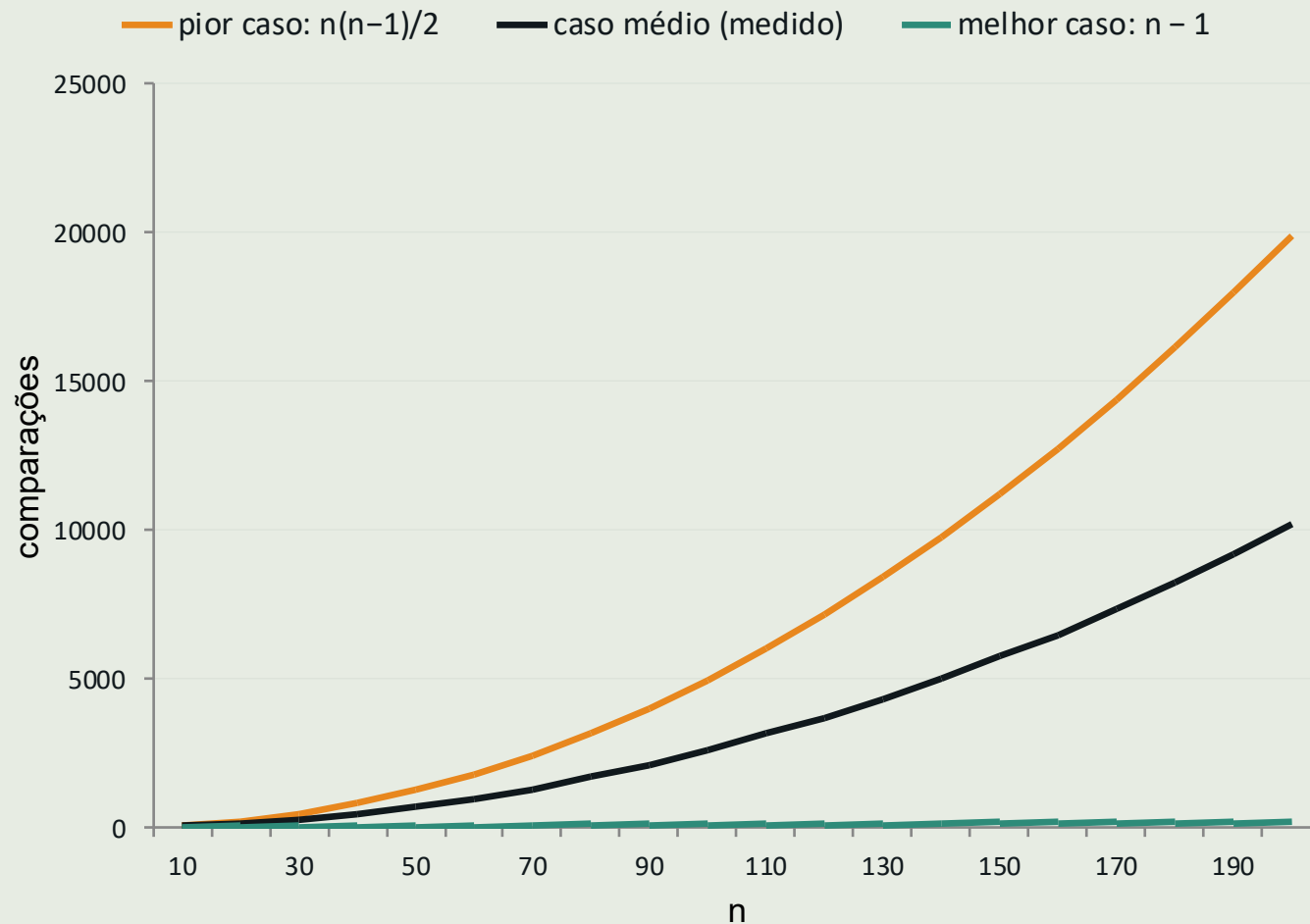
$n - 1$ comparações
 $\rightarrow \Theta(n)$

PIOR CASO: ORDEM INVERSA

5	4	3	2	1	
4	5	3	2	1	1 comp., 1 desl.
3	4	5	2	1	2 comp., 2 desl.
2	3	4	5	1	3 comp., 3 desl.
1	2	3	4	5	4 comp., 4 desl.

$1 + 2 + \dots + (n - 1) = n(n - 1)/2$
 $\rightarrow \Theta(n^2)$

CASO MÉDIO



n	ordenado	aleatório	invertido
10	9	30	45
100	99	2.607	4.950
1.000	999	252.249	499.500

Em média, cada chave anda **metade** da parte ordenada:

$$\approx n^2/4 \rightarrow \Theta(n^2)$$

Metade do pior caso continua quadrático!

DEMO AO VIVO

EXECUTE AGORA cole `python/insertion_sort.py` em `onlineide.pro/playground/python` e rode

```
while j >= 0:
    comparacoes_passada += 1    # conta v[j] > chave
    if not (v[j] > chave):
        break                  # achou o lugar
    v[j + 1] = v[j]             # desloca
    deslocamentos_passada += 1
    j -= 1
v[j + 1] = chave
```

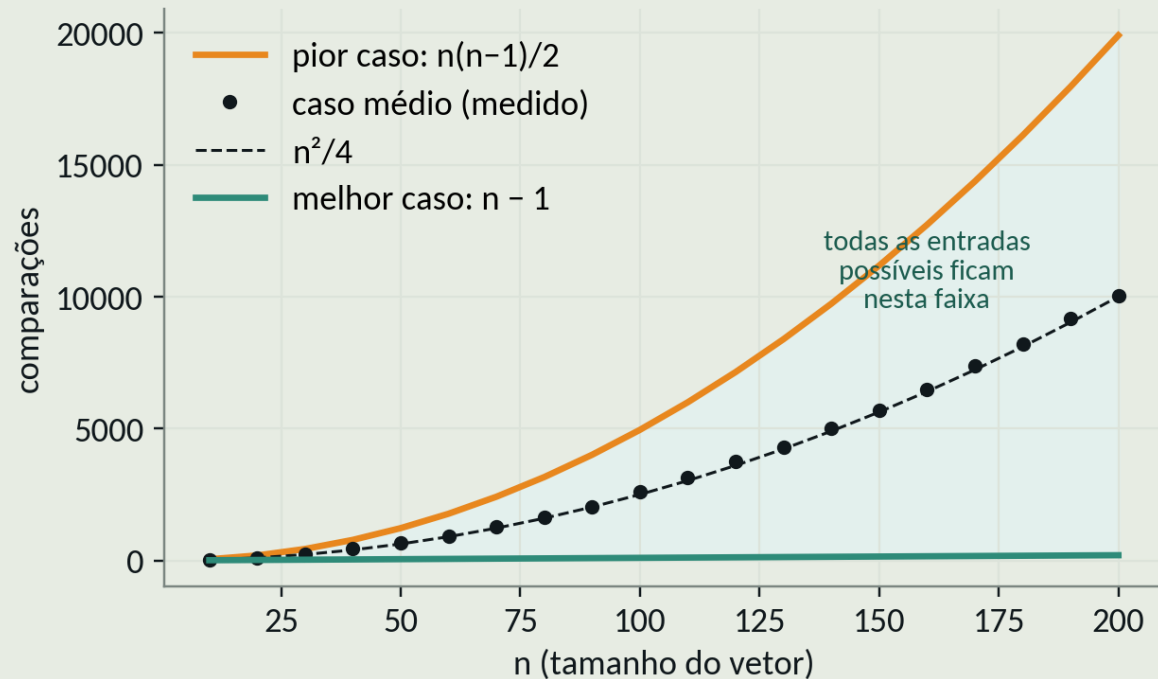
PYTHON

n	ordenado	aleatorio(*)	invertido
10	9	30	45
100	99	2607	4950
1000	999	252249	499500

(*) media de varios vetores embaralhados

A versão com contador é a mesma do slide 29, trocando o "and" do while por um if que conta cada comparação. A versão em C está em `c/insertion_sort.c`.

CASOS NÃO SÃO NOTAÇÕES



CERTO

O **pior caso** do Insertion Sort é $\Theta(n^2)$.

CERTO

O **melhor caso** é $\Theta(n)$.

CERTO

O Insertion Sort é $\mathbf{O}(n^2)$ e $\mathbf{\Omega}(n)$: todas as entradas ficam na faixa.

ERRADO

"O Insertion Sort é $\Theta(n^2)$." — de qual caso?

ERRADO

"O é o pior caso; Ω é o melhor caso."

EXERCÍCIO 7

EXERCÍCIO 7

PAUSE E TENTE · RESPONDA NO CHAT · CERCA DE 2 MIN

a

Aplique o Insertion Sort a **[1, 2, 3, 5, 4]**: quantas **comparações** e quantos **deslocamentos**? Está mais perto do melhor ou do pior caso?

b

Para **$n = 8$** , quantas comparações o Insertion Sort faz no **melhor** caso? E no **pior**?

RESPOSTA 7

RESPOSTA 7

1	2	3	5	4
---	---	---	---	---

chave 2: 1 comp.

1	2	3	5	4
---	---	---	---	---

chave 3: 1 comp.

1	2	3	5	4
---	---	---	---	---

chave 5: 1 comp.

1	2	3	4	5
---	---	---	---	---

chave 4: 2 comp., 1 desl.

(a) **5 comparações, 1 deslocamento**

Quase o melhor caso ($n - 1 = 4$).

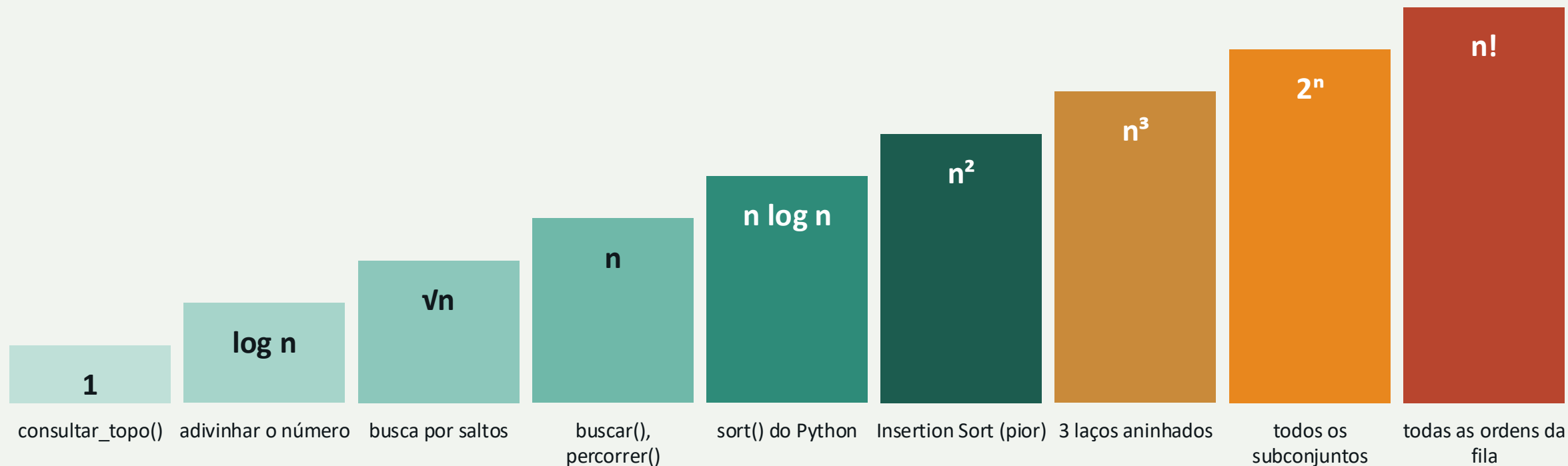
Por isso o `sort()` do Python usa Insertion Sort em trechos pequenos e quase ordenados.

(b) Melhor caso: $n - 1 = 7$. Pior caso: $n(n - 1)/2 = 8 \times 7 / 2 = 28$.

A HIERARQUIA

$$1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < 2^n < n!$$

Leia "<" como "cresce mais devagar que", para n grande.



TEMPO E HARDWARE

Se cada passo levasse 1 nanossegundo...

passos	n = 10	n = 20	n = 50	n = 100
n	10 ns	20 ns	50 ns	100 ns
n ²	100 ns	400 ns	2,5 µs	10 µs
n ³	1 µs	8 µs	125 µs	1 ms
2 ⁿ	1 µs	1 ms	13 dias	4 × 10 ¹³ anos
n!	3,6 ms	77 anos	10 ⁴⁸ anos	10 ¹⁴¹ anos

Idade do universo: $\approx 1,4 \times 10^{10}$ anos.

E um computador 1.000× mais rápido?

passos	maior n em 1 s	ganho
n	$10^9 \rightarrow 10^{12}$	× 1.000
n ²	$31.622 \rightarrow 10^6$	× 31,6
n ³	$1.000 \rightarrow 10.000$	× 10
2 ⁿ	29 → 39	só + 10
n!	12 → 14	só + 2

Trocar de computador multiplica a velocidade por uma **constante**. Trocar de algoritmo pode mudar a **família** — e isso vale muito mais.

EXERCÍCIO 8

EXERCÍCIO 8

PAUSE E TENTE · RESPONDA NO CHAT · CERCA DE 2 MIN

a

Ordene da que cresce mais devagar para a mais rápida: n^2 , 2^n , $n \log n$, 100, $n!$, $\sqrt{n}$, n^3 , $\log n$

b

Um algoritmo 2^n leva **1 s** com **$n = 30$** . Quanto leva com $n = 31$? E com $n = 40$? E se fosse n^2 ?

RESPOSTA 8

RESPOSTA 8

100

$\log n$

$\sqrt{n}$

$n \log n$

n^2

n^3

2^n

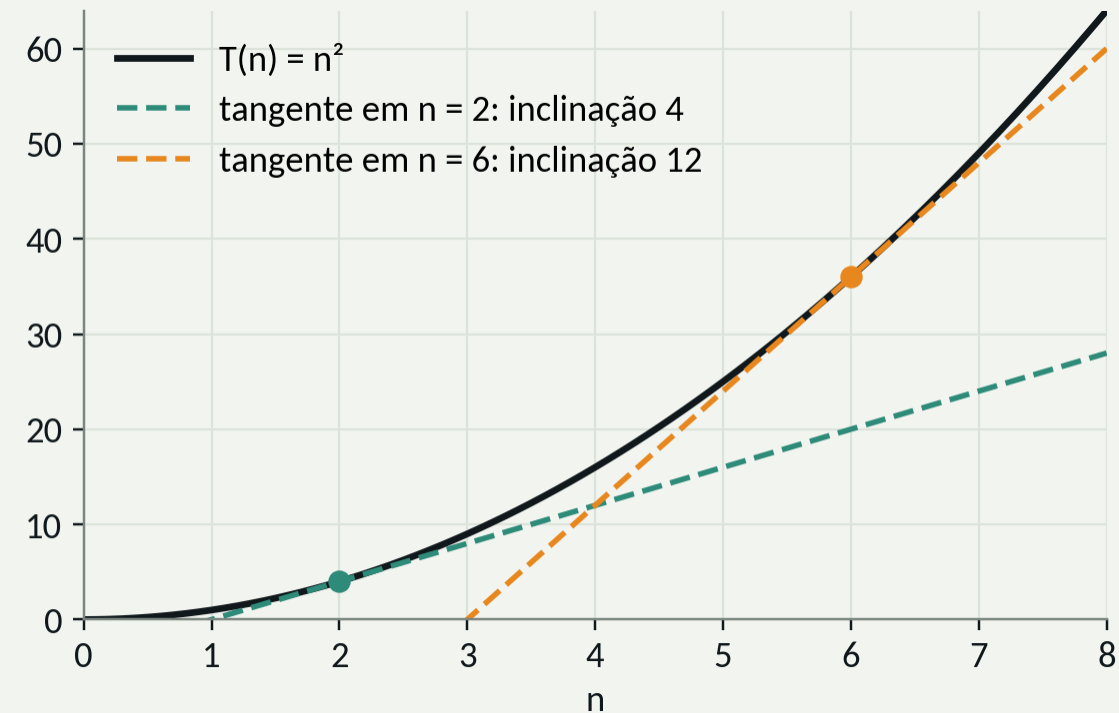
$n!$

2^n : cada +1 em n **dobra** o tempo.

- $n = 31 \rightarrow 2 \text{ s}$
- $n = 40 \rightarrow 2^{10} = 1.024 \text{ s} \approx 17 \text{ minutos}$

n^2 : $(40/30)^2 = 16/9 \rightarrow$ cerca de **1,8 s**

DERIVADA



Derivada = inclinação da tangente: quanto T cresce quando n cresce um pouquinho.

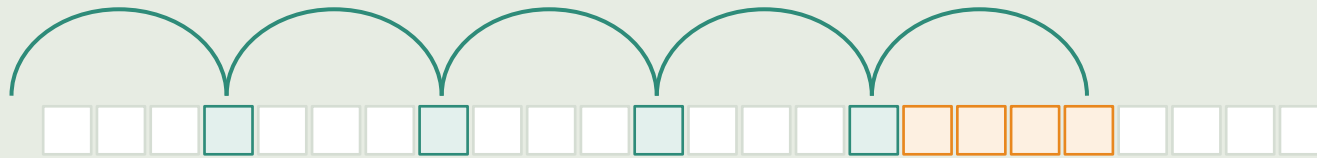
Regra da potência: $d/dn (n^p) = p \cdot n^{p-1}$
Regra da cadeia: $d/dn (u^p) = p \cdot u^{p-1} \cdot u'$

$T(n)$	$T'(n)$
n	1
n^2	$2n$
n^3	$3n^2$
$\sqrt{n} = n^{1/2}$	$1/(2\sqrt{n})$
$1/n = n^{-1}$	$-1/n^2$

Custo marginal
Insertion Sort, pior caso:
 $T(n) = (n^2 - n)/2$
 $T'(n) = n - 1/2 \approx n$

cada elemento a mais custa $\approx n$ comparações extras

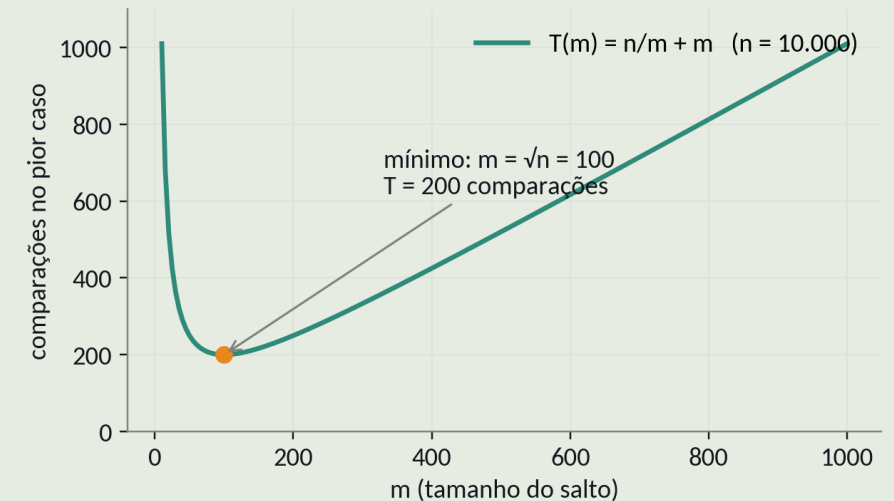
BUSCA POR SALTOS



fase 1: salta de m em m (acesso por índice $O(1)$, Aula 1) · fase 2: busca linear no bloco

- Vetor **ordenado** com n elementos (uma lista de chamada com 10.000 nomes).
- Pior caso: cerca de n/m saltos + m comparações no bloco.
- Salto pequeno: saltos demais. Salto grande: bloco enorme.

$$T(m) = n/m + m \quad \text{Qual } m \text{ é o melhor?}$$



A DERIVADA RESOLVE

$$T(m) = n \cdot m^{-1} + m$$

$$T'(m) = -n \cdot m^{-2} + 1 \quad (\text{potência, } p = -1)$$

$$T'(m) = 0 \rightarrow m^2 = n \rightarrow m = \sqrt{n}$$

$$n = 10.000 \rightarrow m = 100 \rightarrow \mathbf{200 \text{ comparações}}$$

Contra 10.000 da busca linear. Em geral: $T = 2\sqrt{n} \rightarrow O(\sqrt{n})$.

m	pior caso medido	n/m + m
1	10001	10001
10	1010	1010
25	425	425
50	250	250
100	200	200
200	250	250
400	425	425
1000	1010	1010

Testando todos os m de 1 a 1000:

menor pior caso = 200 comparações

obtido com m de 91 a 110 (20 valores empatados)

$\text{raiz}(10000) = 100 \rightarrow 2 * \text{raiz}(n) = 200$

Perto do mínimo, 20 valores de m empatam: a curva é quase plana onde a derivada vale zero.

EXERCÍCIO 9

EXERCÍCIO 9

PAUSE E TENTE · RESPONDA NO CHAT · CERCA DE 2 MIN

a

Busca por saltos com $n = 400$: qual é o melhor salto e quantas comparações no pior caso?

b

E com $n = 1.000.000$? Compare com a busca linear.

c

Regra da cadeia: se $T(n) = (n + 1)^2$, quanto vale $T'(n)$?

RESPOSTA 9

RESPOSTA 9

**$m = 20$
 40 comp.**

$\sqrt{400} = 20$; $T = 400/20 + 20 = 40$ comparações (contra 400 da busca linear).

**$m = 1.000$
 2.000 comp.**

$\sqrt{1.000.000} = 1.000$; $T = 1.000 + 1.000 = 2.000$ comparações (contra 1.000.000).

$2n + 2$

$u = n + 1, u' = 1$: $T'(n) = 2 \cdot (n + 1)^1 \cdot 1 = 2n + 2$.

REGRAS PRÁTICAS

Padrão	Custo
comandos simples	$\Theta(1)$
um laço até n	$\Theta(n)$
laço dentro de laço (ou triangular)	$\Theta(n^2)$
laço que divide ou multiplica por 2	$\Theta(\log n)$
blocos em sequência	soma: fica o maior
chamada $\Theta(m)$ dentro de laço de n	$\Theta(n \cdot m)$

Cuidado com o Python: x in lista, insert(0, x), remove(x), min, sum... cada um esconde um laço de n.

```
for i in range(n):  
    for j in range(n):  
        print(i, j)  
for k in range(n):  
    print(k)
```

$n^2 + n \rightarrow O(n^2)$

```
i = n  
while i > 1:  
    i = i // 2
```

$O(\log n)$

```
for i in range(n):  
    for j in range(i):  
        print(i, j)
```

Gauss $\rightarrow O(n^2)$

REVISÃO PARA A PROVA

Estrutura	Operação	Custo
Lista contígua	obter(pos) · inserir no início	$\Theta(1) \cdot \Theta(n)$
Lista encadeada	inserir no início · obter(pos)	$\Theta(1) \cdot O(n)$
Pilha contígua	empilhar · desempilhar · topo	$\Theta(1)$
Fila encadeada (com fim)	enfileirar · desenfileirar	$\Theta(1)$
Fila sem ponteiro fim	enfileirar	$\Theta(n)$
Conjunto sobre lista	pertence · inserir · remover	$O(n)$
set() do Python	in · add · remove	$O(1)$ em média

PROVA DO GRAU 1 NO LEX

- individual e automática
- cobre os **4 encontros**
- prazo definido pela coordenação

Para estudar:

- apostila, caps. 13 e 16
- cap. 15: 28 exercícios resolvidos, com questões objetivas

PRÁTICA: CAMPO MINADO

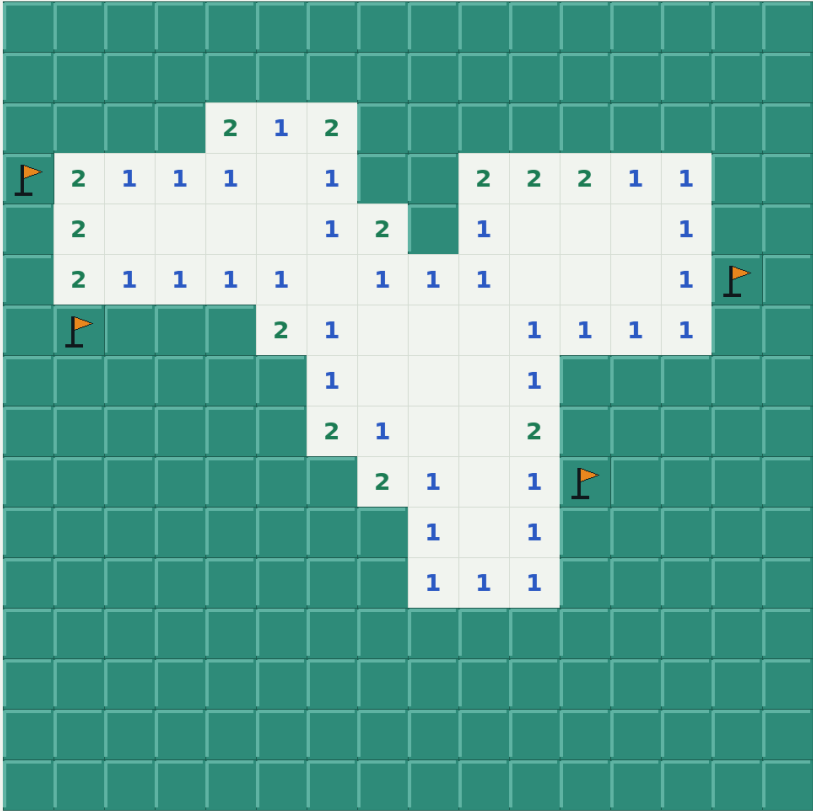
Campo Minado — Estruturas de Dados (Aula 4)

Jogo Análise Ajuda

Minas: 36

Novo jogo

Tempo: 37



Abertura: FILA (em ondas)

Última jogada:
comparacoes conjunto 6

Partida inteira:
comparacoes conjunto 1338
sorteios 46
vizinhos examinados 462
entradas estrutura 68
tamanho max estrutura 14
celulas visitadas 68

Histórico (topo primeiro):
bandeira (10, 12)
bandeira (4, 1)
bandeira (6, 15)
bandeira (7, 2)
revelar (9, 9)

PRÁTICA DE AUTOESTUDO · SEM ENTREGA

Aula	No jogo
1	tabuleiro numa lista linear
2	Fila (ondas) e Pilha (profundidade, desfazer)
3	Conjunto: minas sem repetição, bandeiras
4	contador de operações e análise de custo

Esqueleto: exercicios/campo_minado (12 TODOs)

Solução comentada: solucoes/campo_minado + GABARITO.md — tente antes!

Enunciado: material/ (PDF)

IDE: PyCharm ou Thonny · C: CLion ou Code::Blocks

ATÉ A PRÓXIMA

Medimos algoritmos **contando passos**, não segundos: o que importa é a forma do crescimento.

O , Ω e Θ são teto, piso e crescimento exato; melhor e pior caso são situações dos dados.

Trocar de algoritmo muda a **família**; trocar de computador só muda uma constante.



PRÓXIMOS PASSOS

Apostila e lista do capítulo 15 · Prática com o Campo Minado · Prova do Grau 1 no LEX